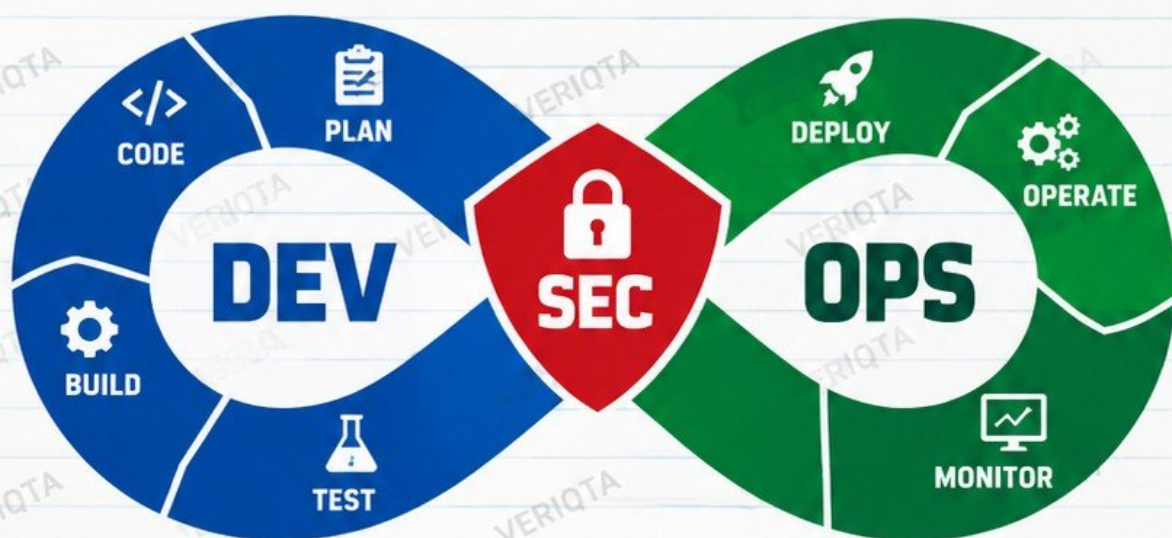


DEVSECOPS BOOTCAMP

A 30-PAGE HANDS-ON GUIDE
FOR JUNIOR ENGINEERS



BUILD SECURE SYSTEMS • AUTOMATE SECURITY • RUN WITH CONFIDENCE



Git



GitHub



Docker



Kubernetes



Terraform



AWS



Security



Monitoring



CLEAR
EXPLANATIONS



PRACTICAL
EXAMPLES



REAL-WORLD
SCENARIOS



BEGINNER TO
JOB READY

YOUR DEVSECOPS JOURNEY STARTS HERE

SAME
DISCIPLINE
HIGHER IMPACT

VERIQTA

EDUCATE • EMPOWER • ENGINEER • EXCEL

Secure
Build
Deploy
Repeat

What Is DevSecOps?

DevOps vs DevSecOps, Development + Security + Operations, shared responsibility, automation, continuous security, and why security belongs throughout the software lifecycle.

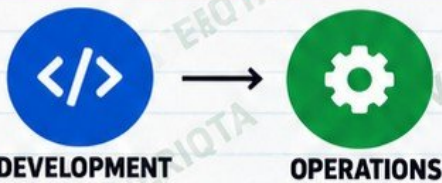
What Is DevOps?

- DevOps is a culture and set of practices that brings Development and Operations together.
- The goal is to build, test and deliver software faster and more reliably.
- It focuses on collaboration, automation and continuous delivery.

What Is DevSecOps?

- DevSecOps is DevOps with security integrated at every stage.
- Security is not a separate team or a final check.
- It is built into the process, tools and culture.
- The goal is to deliver software that is fast, reliable and secure.

DevOps



Focus: Speed, collaboration, automation, reliable delivery.

DevSecOps



Focus: Speed, collaboration, automation and continuous security.

Why Security Belongs Throughout the Lifecycle

- ✓ Find and fix problems early when they are cheaper.
- ✓ Reduce risk in production.
- ✓ Prevent vulnerabilities from being introduced.
- ✓ Meet compliance and regulatory requirements.
- ✓ Build customer trust.
- ✓ Keep your applications and data safe.



Security is everyone's responsibility, not just the security team.

The DevSecOps Lifecycle



Junior Engineer Tip:

DevSecOps is not a tool. It is a mindset. You build security into everything you do.



Common Mistake:

Thinking security is only a final scan before production.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SAME DISCIPLINE
HIGHER IMPACT

The DevSecOps Lifecycle

Security is not a final step. It is integrated into every stage of the lifecycle to build, deliver and operate software that is fast, reliable and secure.



Junior Engineer Tip:

Security is everyone's responsibility. The best DevSecOps teams build security into their culture, not just their tools.



Remember:

A secure pipeline is not slower. It prevents bigger problems later.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

Shift Left, Shift Right, and Continuous Security

Find vulnerabilities early, validate security during deployment, monitor production, and use feedback loops. Security is needed before and after deployment, not just in one stage.



1. SHIFT LEFT

Find and fix issues early

- Include security in design and planning.
- Use threat modeling.
- Follow secure coding practices.
- Run SAST and dependency scans.
- Fix vulnerabilities early when they are cheaper and easier to fix.



Result:

Fewer vulnerabilities reach production.

2. SHIFT RIGHT

Detect and respond in production

- Monitor runs, logs and user activity.
- Detect threats and unusual behavior.
- Use runtime security tools (DAST, monitoring, SIEM).
- Respond to incidents quickly.
- Feed learnings back to improve development, policies and controls.



Result:

Faster detection and response in real situations.

3. WHY BOTH ARE NEEDED

- Shift left alone cannot catch everything.
- Shift right alone means vulnerabilities may already be in production.
- Attackers find new ways to exploit systems.
- You need security before deployment and during runtime.
- Continuous feedback makes the whole system stronger.



Result:

More secure, resilient and reliable applications.



Key Takeaways

- ✓ Security is a continuous process, not a one-time activity.
- ✓ Everyone is responsible for security (developers, operations, security teams).
- ✓ Automate security checks throughout the pipeline.
- ✓ Learn from production incidents and improve.
- ✓ The goal is to prevent, detect, and respond, not just scan.



Common Mistake

Thinking security is only a pre-deployment scan.

- Ignoring runtime security.
- Not acting on security findings.
- No feedback loop after incidents.
- Treating security as someone else's job.



Junior Engineer Tip:

Think of security as a cycle: Build securely → Run safely → Learn from what happens → Make it better.



Remember:

The best systems get stronger over time because they learn from real-world events.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

Security Fundamentals for DevOps Engineers

Understand the key security concepts you need to build, deploy and operate secure systems.

1. CONFIDENTIALITY



Keeps data private and only accessible to authorized people.

Example: Encrypting data, restricting access to secrets.

2. INTEGRITY



Ensures data is accurate, unchanged and trusted.

Example: Using checksums, digital signatures, version control.

3. AVAILABILITY



Ensures systems and data are accessible when needed.

Example: Redundancy, backups, fault tolerance, monitoring.

4. AUTHENTICATION



Verifies who you are. Proves your identity before granting access.

Example: Passwords, SSH keys, MFA (multi-factor authentication).

5. AUTHORIZATION



Determines what you are allowed to do after authentication.

Example: IAM roles, RBAC, file permissions.

6. LEAST PRIVILEGE



Give only the minimum access needed to do a job. Nothing more.

Example: Limited IAM policies, restricted sudo, scoped tokens.

7. ATTACK SURFACE



All the places an attacker can try to gain access to your system.

Example: Open ports, exposed APIs, public cloud resources.

8. VULNERABILITY



A weakness that can be exploited by an attacker.

Example: Outdated software, misconfigurations, insecure code.

9. THREAT



A potential danger that could exploit a vulnerability.

Example: Malicious users, malware, ransomware, insider threats.

10. EXPLOIT



A method or tool used to take advantage of a vulnerability.

Example: SQL injection script, privilege escalation exploit.

11. RISK



The possibility of a threat exploiting a vulnerability and causing harm.

Depends on:

- Likelihood (how likely)
- Impact (how serious)

12. SECURITY CONTROLS



Safeguards that reduce risk by preventing, detecting or responding to threats.

Example: Firewalls, encryption, access control, monitoring, patching, security policies.



Junior Engineer Tip:

Security is everyone's responsibility. Even as a DevOps engineer, you play a key role in building and operating secure systems.



Remember:

A secure system is not just about tools. It's about people, processes and good habits.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

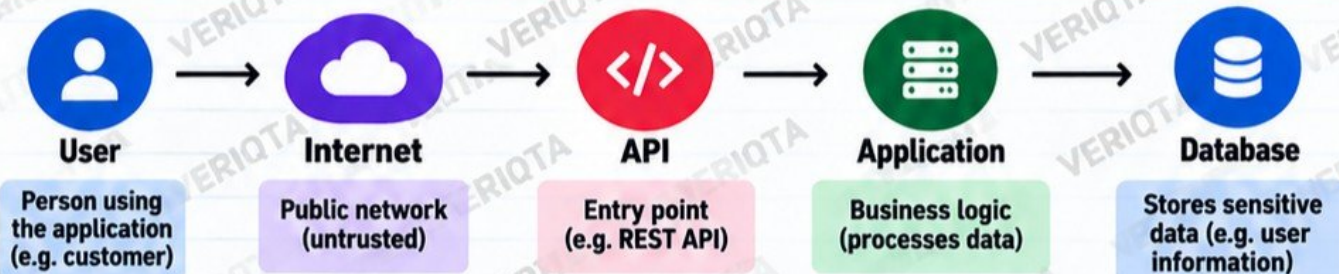
**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Threat Modeling for Junior Engineers

Threat modeling helps you understand what to protect, where the risks are, and how attackers could exploit your system so you can put the right controls in place.

Example Architecture



1. Assets



- Identify what you need to protect.
- Examples: user data, credentials, source code, APIs, infrastructure, reputation.

2. Trust Boundaries



- Define where trust ends and begins.
- Examples: internet to API, API to application, application to database, internal to external systems.

3. Entry Points



- Identify how attackers can interact with your system.
- Examples: web application, APIs, admin interfaces, file uploads, third-party integrations.

4. Threats



- Identify what could go wrong.
- Examples: data theft, unauthorized access, service disruption, malware, privilege escalation.

5. Attack Paths



- Map how an attacker could move through the system.
- Example: compromised user → exploit API → access application → steal data from database.

6. Mitigations



- Define controls to reduce risk.
- Examples: authentication, authorization, input validation, encryption, network segmentation, monitoring, least privilege, patching.

7. STRIDE Threat Categories (Simple Introduction)

S	Spoofing	Pretending to be someone else.	Example: using stolen credentials.
T	Tampering	Modifying data or code.	Example: changing API requests.
R	Repudiation	Denying an action.	Example: user denies performing an action.
I	Information Disclosure	Exposing sensitive information.	Example: leaking user data.
D	Denial of Service	Making a service unavailable.	Example: flooding the API with requests.
E	Elevation of Privilege	Gaining higher access than allowed.	Example: regular user becomes an admin.



Junior Engineer Tip:

Start simple. Draw your system, identify assets and trust boundaries, think like an attacker, and always ask "what could go wrong?"



Remember:

Threat modeling is not a one-time activity. Update it when your system changes.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Secure Git and Repository Practices

Use Git securely to protect your code, your credentials and your organization.

1. Protected Branches



- Protect important branches (e.g. main, production).
- Require pull requests before merging.
- Enforce status checks (tests, security scans).
- Restrict who can push.

Prevents accidental changes and unauthorized code.

2. Pull Requests (PR)



- Use pull requests for all changes.
- Enable code review.
- Discuss and document changes.
- Link issues to PRs.
- Run automated checks before merge.

Improves code quality and catches security issues early.

3. Code Review



- Have at least one reviewer.
- Look for security issues, not just functionality.
- Check for secrets, insecure code and unnecessary permissions.
- Use review checklists.

More eyes, fewer vulnerabilities.

4. Signed Commits (Introduction)



- Sign your commits to prove who made the change.
- Helps with integrity and trust.
- Use GPG or SSH signing.

```
# Example: sign a commit with GPG
git commit -S -m "Add feature"
```

5. .gitignore



- Exclude unnecessary and sensitive files.
- Never commit secrets, env files, build artifacts or local configs.
- Use a proper .gitignore for your project.

```
# Example .gitignore entries
.env
*.log
node_modules/
.idea/
*.pem
```

6. Access Control



- Give access only to people who need it.
- Use teams and roles (read, write, admin).
- Remove access when no longer needed.
- Enable two-factor authentication (2FA).

Follow the principle of least privilege.

7. Repository Permissions



- Use organization-level policies.
- Restrict who can create, delete or modify branches.
- Limit access to sensitive repositories.
- Use branch protection rules.
- Audit permissions regularly.

Keep your repositories safe and well controlled.

8. Never Commit Credentials



- Never store passwords, API keys, tokens or secrets in Git.
- Attackers can find them, even if you delete them later.
- Use environment variables or secret management tools.
- If you accidentally commit a secret, rotate it immediately.

Credentials in Git history can lead to serious security breaches.

Common Mistakes

- Committing .env files.
- Storing API keys in code.
- Using weak repository permissions.
- Skipping code reviews.
- Pushing directly to main.
- Not using .gitignore.

Best Practices Checklist

- ✓ Protect main branch.
- ✓ Use pull requests and code review.
- ✓ Sign commits (optional but recommended).
- ✓ Use .gitignore.
- ✓ Enforce access control and 2FA.
- ✓ Keep secrets out of Git.
- ✓ Regularly audit repository settings.



Junior Engineer Tip:

Assume everything in Git is public. If you would not want it on the internet, do not commit it.



Remember:

Once a secret is in Git history, it can be very hard to remove completely. Prevent it, don't fix it later.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Secrets Management

Keep sensitive information safe, control access, and prevent accidental exposure.

1. What Are Secrets?



Secrets are sensitive pieces of information used by applications and infrastructure.

- API keys
- Passwords
- Tokens (e.g. OAuth, JWT)
- Certificates and private keys
- Environment variables
- Database connection strings
- Cloud credentials (AWS, Azure, GCP)

Secrets give access.
If exposed, attackers can use them to take control.

2. Why Secrets Matter



- Prevent unauthorized access.
- Protect data and infrastructure.
- Maintain compliance.
- Reduce risk of credential abuse.
- Keep your systems and users safe.

A single exposed secret can lead to data breaches, financial loss and loss of trust.

3. Secret Stores



- Store secrets securely instead of in code.
- Control who can access each secret.
- Support rotation and auditing.
- Examples:
 - HashiCorp Vault
 - AWS Secrets Manager
 - Azure Key Vault
 - Google Secret Manager
 - Kubernetes Secrets (encrypted)

Use a dedicated secret store, not plain files.

4. Environment Variables



- Used to pass secrets to applications.
- Keep them out of source code.
- Set them in secure environments (CI/CD, cloud, containers).
- Do not hardcode secrets in scripts or configuration files.

Example (Linux):

```
export DB_PASSWORD=*****
echo $DB_PASSWORD
```

Environment variables are useful, but they are not a secret store. Use them with proper access control.

5. Rotation



- Regularly update secrets (e.g. every 30–90 days).
- Reduce the impact if a secret is exposed.
- Automate rotation where possible.
- Revoke old secrets after rotation.

Rotated secrets limit damage.

6. Accidental Exposure



- Secrets can be exposed by:
 - Committing them to Git.
 - Sharing in chat or email.
- Logging them in applications.
- Storing them in plain text files.
- Using weak access controls.
- Public cloud storage buckets.

Exposed secrets can be found quickly by attackers and bots.

7. Secret Scanning



- Scan your repositories, containers and infrastructure for exposed secrets.
- Use tools like:
 - GitHub secret scanning
 - TruffleHog
 - Gitleaks
 - Detect-secrets
 - Cloud provider scanning tools.

Find and remove exposed secrets immediately.

8. .env File Example

✗ Bad Example (do NOT do this):

```
# .env (NEVER commit this)
DB_PASSWORD=mysecret123
API_KEY=sk_live_abcdef
AWS_ACCESS_KEY=AKIA...
AWS_SECRET_KEY=abcd...
```

✓ Good Example:

```
# .env (safe example)
DB_PASSWORD=
API_KEY=
AWS_ACCESS_KEY=
AWS_SECRET_KEY=
```

9. Best Practices



- Use a dedicated secret store.
- Never commit secrets to Git.
- Use environment variables securely.
- Enable secret scanning.
- Rotate secrets regularly.
- Apply least privilege access.
- Audit who accesses secrets.
- Remove unused secrets.
- Educate your team.

Treat secrets like keys to your house.

10. Common Beginner Mistakes



- Committing .env files with real credentials.
- Hardcoding secrets in code.
- Sharing secrets in chat or screenshots.
- Using weak or default passwords.
- Not rotating secrets.
- Giving too many people access.
- Storing secrets in plain text files.
- Ignoring secret scanning alerts.

Even private repositories can leak. Never assume it will stay private.

11. Real-World Example



Use GitHub secret scanning (automatic) or run a local scan with TruffleHog:

```
trufflehog git \
  file:///path/to/your/repo
```

Or use Gitleaks:

```
gitleaks detect --source .
```

Scan early, scan often.



Junior Engineer Tip:

If you think a value is not sensitive, don't risk it. Store it securely anyway.



Remember:

Accidentally committing an .env file with real credentials is one of the most common and most dangerous mistakes junior engineers make.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Secure Coding and OWASP Fundamentals

Write secure code, understand common vulnerabilities, and know why both developers and DevOps engineers share responsibility for application security.

1. Input Validation



- Always validate and sanitize user input.
- Use allowlists instead of blocklists.
- Check data type, length and format.
- Prevent unexpected or malicious input.

Example (Python):

```
import re
if not re.match(r'^[a-zA-Z0-9_]+$',
username):
    raise ValueError("Invalid input")
```

2. Injection Attacks (SQL, NoSQL, Command)



- Attackers can inject malicious code through user input.
- Common types: SQL injection, NoSQL injection, command injection.
- Use parameterized queries and prepared statements.
- Never build queries using string concatenation.

Example (SQL - Bad vs Good):

```
# Bad (vulnerable)
query = "SELECT * FROM users WHERE
name = '" + name + "'"

# Good (parameterized)
cursor.execute("SELECT * FROM users
WHERE name = %s", (name,))
```

3. Authentication Weaknesses



- Use strong authentication mechanisms.
- Enforce multi-factor authentication (MFA).
- Avoid weak passwords and default credentials.
- Implement account lockout and session management.
- Protect against credential stuffing and brute force attacks.

Example:

- Use MFA.
- Enforce strong password policies.
- Use secure session tokens.

4. Access-Control Failures



- Ensure users can only access what they are allowed to.
- Use role-based access control (RBAC).
- Validate access on the server (not just the UI).
- Prevent horizontal and vertical privilege escalation.

Example:

- Regular users cannot access admin APIs.
- Check authorization for every request.

5. Insecure Configuration



- Avoid default settings.
- Disable unnecessary features and services.
- Keep software, libraries and OS up to date.
- Use secure headers and environment settings.
- Do not expose sensitive information (error messages, stack traces).

Example (HTTP Security Headers):

```
Content-Security-Policy: default-src 'self'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
```

6. Dependency Risks



- Third-party libraries can contain vulnerabilities.
- Use trusted and maintained packages.
- Regularly scan dependencies for known vulnerabilities.
- Remove unused dependencies.
- Keep dependencies updated.

Tools for dependency scanning:

- OWASP Dependency-Check
- npm audit (Node.js)
- pip-audit (Python)
- Snyk, GitHub Dependabot

7. OWASP Top 10 (Key Categories)



- Injection
- Broken Authentication
- Broken Access Control
- Insecure Design
- Security Misconfiguration
- Vulnerable Components
- Identification and Authentication Failures
- Software and Data Integrity Failures
- Security Logging and Monitoring
- Server-Side Request Forgery (SSRF)

8. Shared Responsibility



- Developers write secure code.
- DevOps engineers build secure pipelines, infrastructure and environments.
- Both teams must work together.
- Security is everyone's responsibility.

Secure applications are built, deployed, and operated through collaboration.



Junior Engineer Tip:

Security is not a feature you add at the end. It must be built into the code, pipeline and infrastructure from the start.



Remember:

Insecure code can lead to data breaches, financial loss and damage to user trust. Write, deploy and operate securely.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

SAST, Static Application Security Testing

Analyze source code before execution to find security vulnerabilities early.
Integrate SAST into your CI/CD pipeline to build more secure software.

1. What is SAST?



- SAST (Static Application Security Testing) analyzes source code without running the application.
- Finds security issues early in the development lifecycle.
- Helps developers fix vulnerabilities before they reach production.

SAST looks at the code. It does not execute the application.

2. How SAST Works



- Scans source code, bytecode or compiled code.
- Uses rules and pattern matching to find vulnerabilities.
- Detects risky code constructs (e.g. hardcoded secrets, SQL queries, insecure functions).
- Produces a report with findings, severity levels and remediation advice.

SAST uses static analysis rules to identify potential security issues in your code.

3. Pipeline Integration



- Run SAST automatically in your CI/CD pipeline.
- Fail the build on high or critical issues (optional).
- Publish results as part of build artifacts.
- Integrate with tools like SonarQube, Semgrep, Checkmarx, Fortify or CodeQL.

Automate SAST so every code change is scanned.

4. Common Findings



- Injection vulnerabilities (SQL, command injection).
- Hardcoded secrets (API keys, passwords, tokens).
- Insecure cryptography usage.
- Cross-site scripting (XSS).
- Insecure file handling.
- Use of vulnerable libraries.
- Insecure configuration settings.

SAST helps you find risky code before attackers do.

5. Severity Levels



- **Critical** – immediate fix (required).
- **High** – fix as soon as possible.
- **Medium** – should be fixed.
- **Low** – fix when possible.
- **Info** – not a direct vulnerability, but a best practice suggestion.

Focus on critical and high findings first, but track and fix the rest.

6. False Positives



- SAST can sometimes flag code that is not actually vulnerable.
- This is called a false positive.
- Review findings manually.
- Tune rules to reduce noise.
- Do not ignore real issues because of false positives.

False positives are normal. Investigate and confirm before closing a finding.

7. Remediation



- Review the finding and understand the risk.
- Follow the tool's remediation guidance.
- Apply secure coding practices.
- Test the fix.
- Re-scan to confirm the issue is resolved.

Fix the root cause, not just the symptom.

8. Example: Run SAST in CI (GitHub Actions)

```
name: SAST Scan
on: [push, pull_request]
jobs:
  sast:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Run Semgrep
        uses: returntocorp/semgrep-action@v1
        with:
          config: auto
```

9. Popular SAST Tools

- SonarQube
- Semgrep
- Checkmarx
- Fortify (SAST)
- GitHub CodeQL
- PMD
- ESLint (for JavaScript)
- Bandit (for Python)
- SpotBugs (for Java)

Choose a tool that fits your stack and integrate it into your workflow.



Junior Engineer Tip:

Fixing security issues early is easier and cheaper than fixing them in production.



Remember:

SAST is not a replacement for manual code review or runtime testing. It is one part of a complete security strategy.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Dependency Security and SCA

Find, understand and fix vulnerable dependencies to keep your software supply chain secure.

1. What is SCA?



- SCA (Software Composition Analysis) scans your application's dependencies to find known vulnerabilities.
- Checks open-source and third-party libraries.
- Helps you manage risk in your software supply chain.

SCA tells you what you are using and whether it is safe.

2. Vulnerable Dependencies



- Dependencies can contain security vulnerabilities (CVEs).
- Attackers can exploit vulnerable libraries to compromise your application.
- Even popular libraries can have critical vulnerabilities.

You can have secure code but still be at risk because of a vulnerable dependency.

3. Transitive Dependencies



- Your application depends on libraries, and those libraries depend on other libraries (transitive dependencies).
- A vulnerable transitive dependency can still affect your application.
- You might not use it directly, but it is still part of your software.

Always scan the full dependency tree, not just direct dependencies.

4. CVEs (Common Vulnerabilities and Exposures)



- A CVE is a publicly known security vulnerability.
- Each CVE has a unique ID (e.g. CVE-2024-3094).
- Includes a description, severity score (CVSS) and affected versions.
- Use CVE databases (NVD) to learn more about vulnerabilities.

Example CVE ID:

CVE-2024-3094

Severity: 9.8 (Critical)

Description: Remote code execution in vulnerable library.

5. Version Management



- Use specific versions instead of wide version ranges (e.g. use 1.2.3 instead of ^1.2).
- Regularly update dependencies to get security fixes.
- Review release notes before upgrading.
- Remove unused dependencies to reduce risk.

Good: requests==2.31.0

Risky: requests>=2.0.0

Best: keep dependencies updated and review changes.

6. Dependency Scanning



- Use SCA tools to scan your project for vulnerable dependencies.
- Integrate scanning into your CI/CD pipeline.
- Fail the build on critical vulnerabilities.
- Popular tools: OWASP Dependency-Check, Snyk, Trivy, GitHub Dependabot, npm audit.

Scan early, scan automatically, scan every commit.

7. Remediation



- Update to a secure version.
- Apply patches or use an alternative library.
- If no fix is available, use a workaround (e.g. disable vulnerable feature).
- Remove unnecessary dependencies.
- Re-scan to confirm the issue is resolved.

Track, fix and verify.

8. Example: npm Audit



```
# Scan for vulnerable packages  
npm audit
```

```
# Fix automatically (where possible)  
npm audit fix
```

```
# Fix everything (may include  
breaking changes)  
npm audit fix --force
```

Use similar commands for other ecosystems (pip, Maven, etc).

9. Best Practices



- Scan dependencies regularly.
- Keep dependencies updated.
- Use trusted sources and registries.
- Remove unused dependencies.
- Monitor for new CVEs.
- Use automated tools in CI/CD.
- Set policies for acceptable risk levels.
- Educate your team.

A secure supply chain builds a more secure application.



Junior Engineer Tip:

Always check your dependencies before deploying.
One vulnerable package can put your entire system at risk.



Remember:

Dependency security is everyone's responsibility.
Developers choose the libraries, and DevOps engineers enforce scanning and policies.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

CI/CD Pipeline Security

Secure your pipeline, protect your code, and prevent attackers from reaching production.

1. Pipeline Attack Surface



- CI/CD pipelines are high-value targets for attackers.
- Attackers can inject malicious code, steal secrets, or modify builds.
- Risks exist in code, configuration, third-party tools, runners, and artifacts.
- A compromised pipeline can deploy malicious software to production.

Your pipeline is part of your production environment. Secure it like one.

2. Runners and Build Agents



- Runners execute your pipeline jobs and can be targeted.
- Use trusted, hardened runners (e.g. GitHub-hosted or self-hosted with proper security).
- Keep runners isolated and updated.
- Avoid running untrusted code on privileged runners.
- Use ephemeral runners when possible.

A compromised runner can steal secrets, modify artifacts and access your infrastructure.

3. Credentials and Secrets



- Never hardcode credentials in pipeline files.
- Use a secure secret store (e.g. GitHub Secrets, GitLab CI Variables, AWS Secrets Manager).
- Use least privilege access.
- Rotate credentials regularly.
- Do not expose secrets in logs or build output.

If an attacker gets your pipeline credentials, they can access your entire infrastructure.

4. Permissions and Access Control



- Use least privilege permissions for pipeline jobs.
- Restrict who can trigger pipelines (e.g. only trusted users/branches).
- Use role-based access control (RBAC).
- Limit permissions for tokens (e.g. GitHub Actions GITHUB_TOKEN).
- Review and audit access regularly.

Give only the permissions needed. More permissions = higher risk.

5. Third-Party Actions/Plugins



- Use trusted and well-maintained actions/plugins.
- Review the source code before using third-party components.
- Pin versions (use commit SHAs, not just tags like latest).
- Be aware of supply chain risks in CI/CD tools.
- Regularly update and remove unused actions/plugins.

A malicious or compromised plugin can steal secrets or inject malware into your pipeline.

6. Untrusted Code



- Be careful with pull requests from forks or external contributors.
- Do not run untrusted code with access to secrets.
- Use separate workflows for untrusted code (read-only).
- Validate and scan code before running build steps.
- Use manual approval for sensitive actions.

Never give untrusted code access to your secrets or production environments.

7. Artifacts and Build Outputs



- Artifacts can contain malicious code.
- Scan artifacts for vulnerabilities before deployment.
- Store artifacts in a secure registry (e.g. GitHub Packages, AWS ECR).
- Use artifact signing and verification (e.g. checksum, Sigstore, Cosign).
- Set appropriate retention policies.

Trust but verify. Always scan what you build.

8. Protected Deployment Environments



- Use environment protection rules (e.g. required reviews, manual approvals).
- Restrict who can deploy to production.
- Use separate credentials for different environments (dev, staging, prod).
- Enable audit logs for deployments.
- Implement rollback plans.

Make it hard for attackers to deploy malicious code to production.

9. Best Practices Checklist



- ✓ Use least privilege access.
- ✓ Secure your runners.
- ✓ Store secrets securely.
- ✓ Review third-party actions.
- ✓ Scan code, dependencies and artifacts.
- ✓ Use protected environments.
- ✓ Enable logging and monitoring.
- ✓ Keep your CI/CD tools updated.
- ✓ Educate your team.

A secure pipeline delivers secure software.



Junior Engineer Tip:

Treat your CI/CD pipeline as critical infrastructure, because it can directly impact your production environment.



Remember:

A single misconfiguration in your pipeline can lead to data breaches, supply chain attacks and full system compromise.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

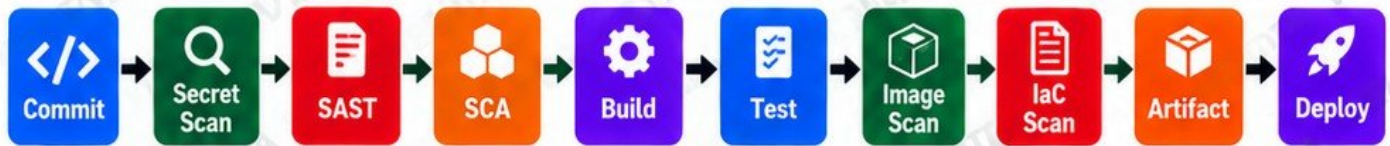
**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Building a DevSecOps Security Pipeline

Integrate security at every stage. Find and fix issues early, automate checks, and ship secure software.

DEVSECOPS SECURITY PIPELINE FLOW



AUTOMATE SECURITY. FAIL FAST. KEEP YOUR SOFTWARE, INFRASTRUCTURE AND USERS SAFE.

1. Commit



- Developers commit code to the repository.
- Triggers the pipeline.
- Starting point for security checks.

New code enters the pipeline. Security checks start immediately.

2. Secret Scan



- Scans code for exposed secrets (API keys, passwords, tokens).
- Prevents credentials from entering the repository.
- Uses tools like GitLeaks, TruffleHog.

Finds hardcoded secrets before they can leak.

3. SAST



- Analyzes source code without running it.
- Finds security flaws like injection, insecure functions, hardcoded values, and more.
- Examples: SonarQube, Sengrep, Checkmarx.

Detects code-level vulnerabilities early.

4. SCA



- Scans third-party libraries and dependencies.
- Finds known vulnerabilities (CVEs).
- Checks for license risks.
- Examples: OWASP Dependency-Check, Snyk.

Identifies vulnerable and risky dependencies.

5. Build



- Compiles the code and creates binaries or artifacts.
- Uses secure build environments.
- Ensures build tools and base images are trusted and up to date.

Produces a build from verified and scanned code.

6. Test



- Runs unit, integration and security tests.
- Includes dynamic tests where possible (e.g. DAST).
- Validates functionality and security controls.

Ensures the application works and is secure.

7. Image Scan



- Scans container images for vulnerabilities.
- Checks OS packages, libraries and configurations.
- Examples: Trivy, Grype, Clair.

Finds vulnerabilities in the container image before deployment.

8. IaC Scan



- Scans Infrastructure as Code (Terraform, CloudFormation, etc.).
- Finds misconfigurations and security risks (e.g. open ports, public buckets).
- Examples: Checkov, TFLint, Terrascan.

Detects risky infrastructure settings.

9. Artifact



- Stores the verified build artifacts (binaries, container images, etc.).
- Ensures artifacts are signed, immutable and from trusted builds.
- Uses artifact registries (e.g. Nexus, ECR, GCR).

Keeps only trusted and verified artifacts.

10. Deploy



- Deploys to staging or production environments.
- Uses protected environments, approvals and policies.
- Monitors for security and compliance after deployment.

Releases secure software to users.



Key Takeaways

- ✓ Security must be integrated, not added at the end.
- ✓ Automate security checks in your CI/CD pipeline.
- ✓ Stop the pipeline when critical security issues are found.
- ✓ Use trusted tools and keep them updated.
- ✓ Monitor and improve your pipeline continuously.



Remember

- ✓ Each security gate checks for different risks.
- ✓ Early detection is faster, cheaper and safer.
- ✓ A secure pipeline protects your code, infrastructure and users.
- ✓ DevSecOps is a shared responsibility across developers, operations and security teams.



Junior Engineer Tip:

Start small. Add secret scanning, SAST and SCA to your pipeline first. Then gradually add image scanning and IaC scanning.



Security is not a step, it is a mindset.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Security Gates and Vulnerability Policies

Define clear policies, set the right thresholds, and handle exceptions properly to keep your software secure without blocking innovation.

1. What are Security Gates?



- Security gates are checkpoints in your pipeline.
- They evaluate security findings and decide if the pipeline should pass or fail.
- They help prevent vulnerable code, dependencies, containers or infrastructure from reaching production.

A security gate enforces your security policy automatically.

2. Pass/Fail Criteria



- Each gate has clear rules for what passes and what fails.
- Rules are based on severity, exploitability, and context.
- Pipelines fail when findings meet or exceed the threshold.
- Pipelines pass when findings are below the threshold or explicitly approved.

No clear pass/fail criteria leads to inconsistent and unreliable security.

3. Severity Thresholds



- Set thresholds for each tool (SAST, SCA, Container Scan, IaC Scan, etc.).
- Common example:
 - **Critical** → Fail
 - **High** → Fail
 - **Medium** → Warn or Fail (depending on risk)
 - **Low** → Allow (usually)
 - **Info** → Allow

Focus on what is truly risky. Not every finding is a production risk.

4. Exceptions



- Sometimes a finding cannot be fixed immediately.
- Exceptions allow the pipeline to pass with approval.
- Every exception must have:
 - A reason
 - An owner
 - A defined expiration date
 - A plan to fix

Exceptions are not permanent fixes. They are temporary approvals with accountability.

5. False Positives



- Tools can flag issues that are not real vulnerabilities.
- These are called false positives.
- Review findings manually to confirm.
- Tune your tools and rules to reduce noise.
- Do not disable security checks to remove false positives.

Investigate, confirm, and document false positives. This keeps trust in the security process.

6. Risk Acceptance



- Some risks may be accepted due to business need, technical constraints, or low exploitability.
- Risk acceptance requires:
 - Documented justification
 - Approval from the right stakeholders
 - A defined expiration date
 - Ongoing review

Accepted risks are still risks. They must be tracked and reviewed regularly.

7. Expiration of Exceptions



- All exceptions must expire.
- Set a clear expiration date (e.g. 30, 60, or 90 days).
- Re-evaluate the risk before renewing.
- If the issue is still not fixed, escalate it.

No expiration date = forgotten risk.

8. Why Not Block Everything?



- Blindly blocking every finding creates problems:
 - Slows down development
 - Creates alert fatigue
 - Leads to ignored warnings
 - Reduces trust in security tools
 - Can block low-risk or irrelevant findings
 - Discourages teams from using security checks

Be smart. Block what matters, manage the rest.

9. Best Practices



- Define clear policies for each tool.
- Use severity-based thresholds.
- Review and handle false positives.
- Document and track exceptions.
- Set expiration dates.
- Regularly review accepted risks.
- Continuously tune your tools.
- Educate your team.
- Balance security with productivity.

A good security policy protects your business and keeps your team moving.



Junior Engineer Tip:

Start with reasonable thresholds and improve over time. Learn from real findings and adjust your policy as your environment matures.



Remember:

The goal is not zero findings. The goal is controlled, well-understood risk.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Secure Build Environments

A secure build environment ensures your code is built safely, reliably and without exposing sensitive data or being tampered with by attackers.

1. What is a Build Environment?



- A build environment is the system that compiles your code, resolves dependencies and creates build artifacts (e.g. binaries, container images).
- It includes build tools, runtimes, dependencies and configurations.
- If an attacker compromises your build environment, they can inject malicious code into your software.

A compromised build environment can produce compromised software.

2. Isolated Builds



- Run builds in isolated environments (containers, VMs, or dedicated runners).
- Prevent access to other systems and sensitive networks.
- Use network restrictions (egress control).
- Ensure builds are repeatable and clean (no leftovers).

Isolation reduces the blast radius if something goes wrong.

3. Trusted Builders



- Use trusted, maintained build images and tools.
- Only use official sources (e.g. official Docker images, verified tool downloads).
- Keep build tools updated and patched.
- Avoid using untrusted or community images without verification.

You are only as secure as the tools you trust.

4. Build Permissions



- Use least privilege permissions for build jobs.
- Separate permissions for build, test and deploy.
- Avoid giving build systems unnecessary cloud or production access.
- Use short-lived credentials (e.g. OIDC, temporary tokens).

Build environments should have only the permissions they need.

5. Ephemeral Runners



- Use ephemeral (short-lived) runners that are destroyed after each build.
- Prevents attackers from maintaining persistence.
- Ensures a clean environment for every build.
- Popular options: GitHub Actions (hosted runners), GitLab ephemeral runners, self-hosted runners in containers.

Ephemeral runners reduce the risk of compromise.

6. Dependency Integrity



- Verify the integrity of dependencies (checksums, signatures).
- Use trusted package registries (e.g. PyPI, Maven Central, NPM official).
- Consider dependency pinning (specific versions).
- Use tools like SCA (Software Composition Analysis) to scan for vulnerable dependencies.

Make sure you are using the dependencies you expect.

7. Reproducibility Concepts



- A reproducible build produces the same output from the same input, every time.
- Helps detect tampering.
- Makes it easier to verify integrity.
- Use version control, pinned dependencies and consistent build environments.

Reproducible builds increase trust and security.

8. Protecting Build Credentials



- Never hardcode credentials in build scripts.
- Use a secure secret store (e.g. GitHub Secrets, GitLab CI Variables, AWS Secrets Manager).
- Use short-lived tokens and rotate regularly.
- Mask secrets in logs.
- Ensure secrets are not exposed in build artifacts or error messages.

Build credentials are high-value targets. Keep them safe.

9. Best Practices Checklist



- ✓ Use isolated and ephemeral build environments.
- ✓ Trust and verify build tools.
- ✓ Apply least privilege permissions.
- ✓ Verify dependency integrity.
- ✓ Keep build environments updated.
- ✓ Use short-lived credentials.
- ✓ Prevent secret leakage in logs.
- ✓ Make builds reproducible.
- ✓ Monitor and audit build activity.

A secure build environment produces secure software.



Junior Engineer Tip:

Treat your build environment like production. If an attacker can control your build, they can control what you ship.



Remember:

Compromise at build time can be invisible to users, but the impact can be massive.



@veriqta



veriqta



veriqta



veriqta



@veriqta



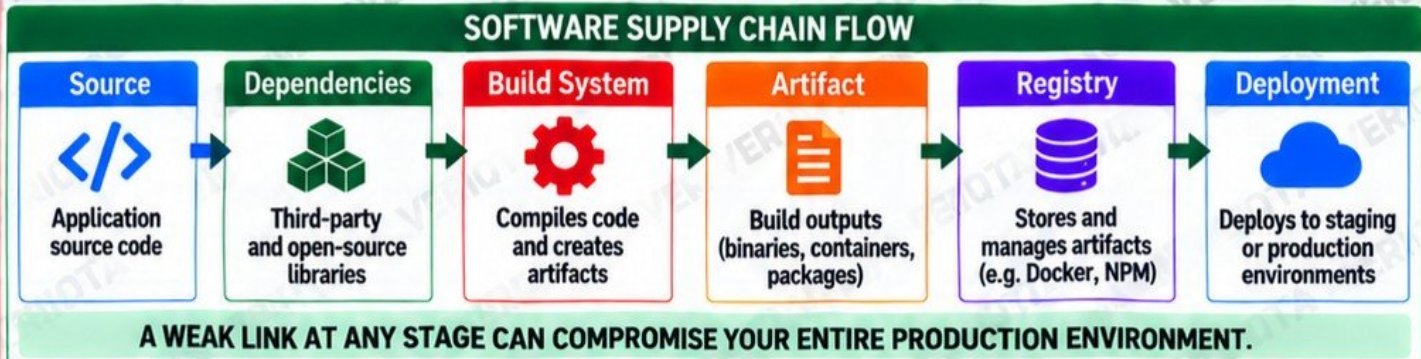
@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Software Supply Chain Security

Your software goes through multiple stages before it reaches production.
A compromise at any stage can affect your entire system.



1. Source (Code Repository)



- Contains your application source code.
- If an attacker gains access, they can inject malicious code, backdoors, or steal secrets.
- Risks include compromised accounts, unprotected branches, or malicious pull requests.

A compromised source repository can introduce malicious code that flows through the entire pipeline.

2. Dependencies



- Your application relies on third-party and open-source libraries.
- If a dependency is compromised, it can introduce vulnerabilities or malicious code.
- Risks include vulnerable packages, typosquatting (fake packages), and unmaintained libraries.

A single vulnerable dependency can give attackers a foothold in your application.

3. Build System



- Compiles your code, runs tests, and creates artifacts (e.g. binaries, container images).
- If the build system is compromised, attackers can modify the build output or inject malicious code.
- Risks include untrusted build environments, excessive permissions, or compromised runners.

A compromised build system can produce malicious artifacts even if your source code is clean.

4. Artifact



- The output of the build process (binaries, container images, packages).
- If an artifact is tampered with, it can contain malware, hidden backdoors, or vulnerable components.
- Risks include lack of signing, no integrity checks, or untrusted build outputs.

A tampered artifact can be deployed to production and give attackers direct access to your systems.

5. Registry



- Stores and distributes artifacts (e.g. Docker Hub, ECR, GCR, NPM).
- If the registry is compromised, attackers can replace legitimate images or packages with malicious ones.
- Risks include weak access controls, public repositories, or compromised credentials.

A compromised registry can deliver malicious artifacts to your deployment environments.

6. Deployment



- Deploys artifacts to staging or production environments.
- If the deployment process is compromised, attackers can gain access to your live infrastructure.
- Risks include excessive permissions, unprotected environments, or lack of verification before deployment.

A compromised deployment can put malicious code directly into production, affecting your users.



Junior Engineer Tip:

Secure every stage, not just production. The whole supply chain matters.



Remember:

A supply chain attack can happen at any stage. Always verify, monitor, and use least privilege.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

SBOM, Software Bill of Materials

Know what is inside your software. An SBOM lists all components, versions and dependencies so you can find and fix risks faster.

1. What is an SBOM?



- An SBOM (Software Bill of Materials) is a machine-readable list of all components in your software.
- It tells you what is inside your application (open-source libraries, frameworks, dependencies, modules, etc.).
- It does not include your source code, but it lists the building blocks used to create it.

An SBOM gives you visibility, which is the first step to security.

2. What an SBOM Records



- Component names (e.g. library, framework)
- Versions (e.g. 1.2.3)
- Unique identifiers (e.g. package URL, hashes)
- Dependencies (direct and transitive)
- Supplier information (e.g. open-source project)
- License information
- Checksums for integrity verification

An SBOM is a complete inventory of your software components.

3. Components and Dependencies



- Components can be direct dependencies (used directly in your code).
- They can also be transitive dependencies (dependencies of dependencies).
- Each component has a specific version.
- Dependencies form a tree structure.
- Even small components can introduce big risks.

You are responsible for both direct and transitive dependencies.

4. Vulnerability Investigation



- An SBOM helps you quickly identify if a vulnerable component is used in your software.
- When a new CVE is announced, you can check your SBOM to see if you are affected.
- You can trace where the component is used and plan a fix.
- It reduces the time to respond to security issues.

With an SBOM, you can find and fix vulnerabilities faster.

5. CycloneDX vs SPDX

CycloneDX

- Open standard for SBOM.
- Widely used in DevOps and security tools.
- Supports multiple formats (JSON, XML).
- Includes dependency relationships, licenses, and metadata.
- Good support in CI/CD tools (e.g. OWASP Dependency-Check, CycloneDX CLI).

SPDX

- Open standard by the Linux Foundation.
- Widely adopted in open-source projects.
- Supports multiple formats (JSON, XML, RDF, tag-value).
- Focuses on licensing and component information.
- Used in many industry tools and ecosystems.

Both CycloneDX and SPDX are industry standards. Choose the one that fits your tools and requirements.

6. Why It Matters



- Helps you manage risk.
- Improves supply chain security.
- Required by many organizations and regulations (e.g. U.S. Executive Order, EU regulations).
- Builds trust with customers.
- Helps with license compliance.
- Makes incident response faster and easier.

You cannot secure what you cannot see.

7. Example SBOM (CycloneDX JSON)

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.5",
  "components": [
    {
      "name": "spring-core",
      "version": "5.3.30",
      "type": "library",
      "purl": "pkg:maven/org.springframework/spring-core@5.3.30"
    }
  ]
}
```

8. Best Practices



- Generate an SBOM for every build.
- Keep SBOMs with your artifacts (e.g. in your container registry).
- Automatically scan SBOMs for vulnerabilities.
- Update dependencies regularly.
- Review licenses and compliance.
- Use tools like CycloneDX, SPDX, Syft, or Trivy.
- Integrate SBOM generation into your CI/CD pipeline.

Make SBOMs a standard part of your development process.

9. Key Takeaways



- An SBOM lists all components in your software.
- It includes names, versions, dependencies, licenses and more.
- It helps you find and fix vulnerabilities quickly.
- CycloneDX and SPDX are the main open standards.
- SBOMs improve security, compliance and trust.
- Knowing what is inside your software matters.

Visibility leads to a more secure and resilient software supply chain.



Junior Engineer Tip:

Start generating SBOMs early. The sooner you know what is inside your software, the easier it is to manage risk.



Remember:

A missing or outdated SBOM can leave you unaware of serious risks in your software.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Artifact Signing, Provenance, and Attestation

Trust what you build. Verify what you deploy. Prevent tampering with cryptographic signatures, provenance and attestations.

1. What are Build Artifacts?



- Artifacts are the output of the build process (binaries, container images, packages).
- They are used in later stages (testing, deployment, etc.).
- If an artifact is modified or replaced, it can introduce malicious code into production.
- Artifacts must be trusted, verified and protected.

An artifact is only valuable if you can trust its integrity and origin.

2. Checksums



- A checksum is a unique hash value (e.g. SHA256) generated from a file.
- Used to verify that a file has not been modified.
- Common algorithms: MD5 (weak), SHA1 (deprecated), SHA256 (recommended).
- If the file changes, the checksum will be different.

Checksums detect accidental or malicious changes to artifacts.

3. Digital Signatures



- A digital signature uses cryptography (private key) to sign an artifact.
- Anyone with the public key can verify the signature.
- Confirms the artifact is from a trusted source and has not been tampered with.
- Common tools: GPG, Sigstore (cosign), Notation.

Signatures prove who created the artifact and that it is unchanged.

4. Provenance



- Provenance describes how an artifact was built.
- It includes details like:
 - Source code repository
 - Build system and version
 - Build environment
 - Build inputs and parameters
- Helps you trace the origin of an artifact and detect suspicious builds.

Provenance gives you a clear history of how an artifact was created.

5. Attestation



- An attestation is a signed statement about an artifact (e.g. provenance, security checks, compliance).
- It is verifiable and tamper-resistant.
- Examples: in-toto attestations, Sigstore attestations.
- Can include SBOM, test results, vulnerability scan results, etc.

Attestations provide evidence that an artifact meets defined security and quality requirements.

6. SLSA Introduction



- SLSA (Supply-chain Levels for Software Artifacts) is a framework from Google.
- Defines levels (0-4) for increasing build security and provenance.
- Higher levels provide stronger guarantees (e.g. hermetic builds, verifiable provenance).
- Widely used in modern DevSecOps to reduce supply chain attacks.

SLSA helps organizations build and verify software with higher levels of trust.

7. Verification Before Deployment



- Always verify artifact checksums and signatures before deployment.
- Validate provenance and attestations.
- Use trusted registries and enforce signature verification (e.g. cosign, Notation).
- Reject artifacts that are unsigned, from unknown sources, or have invalid attestations.

Do not deploy what you cannot verify.

8. Example: Signing and Verifying (cosign)

```
# Sign a container image
cosign sign myapp:1.0.0

# Verify the signature
cosign verify myapp:1.0.0

# Check attestation (example)
cosign verify-attestation \
--type slsa-provenance myapp:1.0.0
```

9. Key Takeaways



- Use checksums to detect changes.
- Use digital signatures to verify identity and integrity.
- Track provenance to know how artifacts were built.
- Use attestations to prove security, compliance and quality.
- Adopt SLSA to improve supply chain security.
- Always verify artifacts before deployment.

A secure software supply chain starts with trust, verification and visibility.



Junior Engineer Tip:

Make artifact signing and verification a mandatory part of your pipeline. Automate it, don't rely on manual checks.



Remember:

Unsigned or unverified artifacts can introduce malicious code into production.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Infrastructure as Code Security

Write secure infrastructure. Prevent misconfigurations. Protect your cloud environments.
IaC gives you speed and consistency, but security must be built in.

1. What is Infrastructure as Code (IaC)?



- IaC means managing infrastructure using code instead of manual configuration.
- Tools include Terraform, CloudFormation, Pulumi, Azure Bicep, and more.
- Your infrastructure is defined in version-controlled files.
- It enables automation, repeatability and consistency.
- But insecure IaC can quickly deploy vulnerable infrastructure at scale.

IaC is powerful. Secure IaC is essential for safe and reliable environments.

2. Misconfiguration Scanning



- Scan IaC code for security misconfigurations before deployment.
- Tools: Checkov, tfsec, Terrascan, Trivy, Bridgecrew (Prisma Cloud), etc.
- Detect issues like public resources, open security groups, weak encryption, etc.
- Integrate scanning into your CI/CD pipeline.

Scan early to catch misconfigurations before they reach your environment.

3. Insecure Defaults



- Many cloud services have insecure defaults (e.g.: public S3 buckets, open security groups).
- IaC can unintentionally deploy these defaults if not reviewed.
- Always explicitly configure security settings (encryption, access controls, logging).
- Do not rely on default values.

Never assume the default is secure. Be explicit in your configuration.

4. Public Exposure



- IaC can accidentally expose resources to the internet (e.g. public S3 buckets, public databases, open ports).
- Check for 0.0.0.0/0 access in security groups.
- Ensure load balancers, storage and databases are not publicly accessible unless required.

Unintended public exposure is one of the biggest IaC security risks.

5. Encryption



- Enable encryption for data at rest (e.g. S3, EBS, RDS).
- Use TLS for data in transit.
- Manage encryption keys properly (e.g. KMS).
- Avoid hardcoding keys or secrets in IaC code.
- Ensure encryption is enabled for logs, backups and sensitive data stores.

Encrypt by default. Protect your data everywhere.

6. IAM Mistakes



- IaC can create overly permissive IAM roles and policies.
- Avoid using * (full access) unless absolutely necessary.
- Follow the principle of least privilege.
- Use managed policies where possible.
- Regularly review and audit IAM permissions.

Over-permissioned IAM roles increase the blast radius of an attack.

7. Policy Checks



- Use policy as code to enforce security standards.
- Tools: Open Policy Agent (OPA), Sentinel (Terraform), AWS Config Rules, or custom policies.
- Check for compliance with security and organizational requirements (e.g. no public resources, encryption enabled).
- Block insecure changes in CI/CD pipelines.

Policy checks help you enforce security consistently and automatically.

8. Code Review



- Always review IaC code before merging.
- Look for security misconfigurations, hardcoded secrets, and unnecessary exposures.
- Use pull requests and require reviews from experienced engineers.
- Combine automated scanning with human review.

Human review catches what tools might miss.

9. Key Takeaways



- Treat IaC code like application code.
- Scan for misconfigurations before deployment.
- Avoid insecure defaults.
- Prevent unintended public exposure.
- Enable encryption.
- Follow least privilege for IAM.
- Use policy checks and code review.
- Automate security in your IaC workflow.

Secure IaC builds a stronger, more resilient cloud environment.



Junior Engineer Tip:

Start with a small Terraform project. Scan it with a tool like Checkov and fix the findings. Make security a habit from the beginning.



Remember:

A single misconfiguration in IaC can expose your entire environment to the internet.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Docker Security Fundamentals

Build and run secure containers. Reduce risk from image to runtime.
A secure container starts with the right image and follows best practices at every layer.

1. Trusted Base Images



- Use official and trusted base images (e.g. Docker Official Images).
- Prefer images from verified publishers.
- Check image provenance and signatures when available.
- Avoid unknown or community images without maintenance.
- Keep base images updated regularly.

A trusted base image reduces your attack surface from the start.

2. Minimal Images



- Use minimal images (e.g. distroless, alpine).
- Include only what your application needs.
- Smaller images have fewer vulnerabilities.
- Remove unnecessary packages, tools and files.
- Use multi-stage builds to keep the final image clean.

Minimal images = smaller size, fewer vulnerabilities, lower risk.

3. Image Scanning



- Scan images for known vulnerabilities (CVEs).
- Use tools like Trivy, Gype, Docker Scout or Snyk.
- Integrate scanning in your CI/CD pipeline.
- Scan both base images and your custom images.
- Fail builds if critical or high severity vulnerabilities are found (based on your policy).

Always scan before you run and before you push.

4. Non-Root Containers



- Do not run containers as root.
- Use a non-root user in your Dockerfile (USER instruction).
- Limit privileges to only what is required.
- This reduces the impact if a container is compromised.
- Combine with read-only filesystem where possible.

A non-root container limits damage and improves security.

5. Secrets Management



- Never hardcode secrets in Dockerfiles or images.
- Use Docker secrets, environment variables, or external secret managers (e.g. AWS Secrets Manager, HashiCorp Vault).
- Avoid copying .env files into images.
- Ensure secrets are not visible in image layers or logs.

Keep secrets out of your images and manage them securely.

6. Capabilities



- Drop unnecessary Linux capabilities.
- Use --cap-drop to remove default capabilities.
- Add only the specific capabilities your application needs (--cap-add).
- Avoid privileged containers (--privileged).
- Use security profiles like seccomp and AppArmor.

Least privilege keeps containers safer.

7. Filesystem Permissions



- Use read-only filesystems where possible (--read-only).
- Set proper file and directory permissions.
- Avoid writing to system directories.
- Use separate directories for application data.
- Clean up temporary files in your image build.

Restricting filesystem access reduces the impact of an attack.

8. Image Tags



- Avoid using the latest tag in production.
- Use specific version tags (e.g. 1.2.3).
- Version tags make builds reproducible and auditable.
- Regularly update to newer secure versions.
- Use image digests (SHA256) for stronger immutability.

Specific tags give you control, consistency and security.

9. Registry Security



- Use trusted and private registries (e.g. Docker Hub, ECR, GCR, ACR, or self-hosted).
- Enable authentication and access control (RBAC).
- Scan images in the registry.
- Use image signing and verify signatures when pulling.
- Set policies to prevent pushing unscanned or vulnerable images.

A secure registry prevents malicious and vulnerable images from reaching production.



Junior Engineer Tip:

Security is a journey. Start with small improvements and make them part of your standard Docker workflow.



Remember:

A container is only as secure as the image, configuration and runtime environment.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Container Image Vulnerability Management

Find, assess, and fix vulnerabilities in your container images. Reduce risk with continuous scanning, timely updates, and realistic security practices.

1. What are CVEs?



- CVEs (Common Vulnerabilities and Exposures) are publicly disclosed security vulnerabilities with unique IDs (e.g. CVE-2024-1234).
- They describe weaknesses in software packages, libraries or base images.
- Each CVE has details about the impact, affected versions and possible fixes.
- Container images often include many packages, so they can inherit CVEs from the base image and installed packages.

CVEs give you visibility into known risks in your images.

2. Package Vulnerabilities



- Container images contain OS packages (e.g. apt, yum) and language packages (e.g. pip, npm, go modules).
- Each package can have its own vulnerabilities.
- Vulnerabilities can exist in direct dependencies and transitive dependencies (dependencies of dependencies).
- Tools like Trivy, Gype, Docker Scout or Snyk scan images and list vulnerable packages.

More packages = larger attack surface and higher risk.

3. Severity vs Exploitability



- Severity (CVSS score) tells you how serious a vulnerability is (Critical, High, Medium, Low).
- Exploitability tells you how easy it is to actually exploit in real-world scenarios.
- A high severity vulnerability may not be easily exploitable in your environment.
- A lower severity vulnerability might be more relevant if it is easily exploitable or publicly exploited.

Prioritize based on both severity and real exploitability, not just the score.

4. Base-Image Updates



- Most vulnerabilities come from the base image (e.g. ubuntu, alpine, amazonlinux).
- Keep base images updated and use actively maintained and minimal images.
- Regularly check for new base image releases.
- Rebuild your images when a new, patched base image is available.

An outdated base image can introduce hundreds of vulnerabilities.

5. Rescanning



- Scan images regularly, not just once during build.
- Rescan when:
 - A new base image is released
 - New CVEs are published
 - Dependencies are updated
 - Your security policies change
- Use automated scanning in your CI/CD pipeline and container registry.

Continuous scanning helps you catch new vulnerabilities early.

6. Remediation



- Update base images and packages to patched versions.
- Remove unnecessary packages.
- Use minimal images (e.g. alpine, distroless).
- Replace vulnerable dependencies in your application.
- Rebuild and redeploy the image after fixes.
- For issues that cannot be fixed immediately, document and track them with a risk acceptance process.

Fix what you can, mitigate what you cannot, and always track the risk.

7. Why "0 Vulnerabilities" Is Unrealistic



- New vulnerabilities are discovered every day.
- Many are in underlying OS packages or transitive dependencies.
- Some vulnerabilities may not have a fix yet.
- Removing every package is not possible because images need packages to function.
- A "0 vulnerabilities" policy can lead to wasted effort, blocked releases, and a false sense of security.

The goal is risk reduction, not perfection.

8. Best Practices



- Use trusted, minimal base images.
- Scan early and scan often.
- Keep images and dependencies updated.
- Automate scanning in CI/CD and registries.
- Prioritize based on risk and exploitability.
- Track and remediate findings.
- Monitor for new CVEs affecting your images.
- Have a clear vulnerability management policy.

A mature process keeps your containers secure over time.

9. Key Takeaways



- Container images contain many packages and can inherit vulnerabilities.
- Use scanning tools to find CVEs and vulnerable packages.
- Consider both severity and exploitability.
- Keep base images updated and rescan regularly.
- Remediate or mitigate findings and track them.
- "0 vulnerabilities" is not realistic — focus on continuous risk reduction.

Security is a journey. Continuous improvement keeps you ahead of threats.



Junior Engineer Tip:

Integrate image scanning into your CI/CD pipeline and registry. Make it a habit to check and update your base images regularly.



Remember:

New vulnerabilities will always be discovered. Stay informed, stay updated, and keep improving.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Kubernetes Security Fundamentals

Secure your Kubernetes cluster, workloads and data. Understand the key security controls that reduce risk and prevent common attacks.

1. Pods Security



- Pods run your containers and share the same network and storage context.
- Use minimal and trusted images.
- Apply security contexts (runAsNonRoot, readOnlyRootFilesystem).
- Limit resource access and avoid privileged containers.
- Treat pods as untrusted by default.

A compromised pod can be the first step to a full cluster breach.

2. Service Accounts



- Service accounts provide identity for pods to access the Kubernetes API.
- Use dedicated service accounts for each application.
- Follow the principle of least privilege.
- Avoid using the default service account.
- Regularly audit and review permissions.

A compromised service account can give an attacker control over your cluster.

3. RBAC (Role-Based Access Control)



- RBAC controls who can do what in your cluster.
- Use roles and role bindings to grant only the required permissions.
- Avoid cluster-admin unless absolutely necessary.
- Regularly review and clean up unused permissions.
- Use separate RBAC for users, service accounts and workloads.

Over-permissive access is one of the biggest Kubernetes security risks.

4. Secrets Management



- Store sensitive data using Kubernetes Secrets.
- Avoid hardcoding secrets in images or manifests.
- Use external secret managers (e.g. AWS Secrets Manager, HashiCorp Vault).
- Enable encryption at rest for etcd.
- Rotate secrets regularly.

Secrets in plain text can be easily stolen and lead to full compromise.

5. Namespaces



- Namespaces provide isolation between teams and applications.
- Use namespaces to separate environments (dev, test, prod).
- Apply resource quotas and limits per namespace.
- Use network policies to control cross-namespace communication.
- Do not rely on namespaces as a complete security boundary.

Namespaces reduce blast radius, but they are not a full security boundary.

6. Security Contexts



- Define security settings for containers and pods.
- Run containers as non-root users (runAsNonRoot).
- Set readOnlyRootFilesystem where possible.
- Drop unnecessary Linux capabilities (e.g. NET_ADMIN).
- Use seccomp, AppArmor or SELinux profiles.

Proper security contexts limit the damage if a container is compromised.

7. Admission Controls



- Admission controllers enforce security policies before objects are created in the cluster.
- Use tools like Pod Security Admission (PSA), OPA/Gatekeeper or Kyverno.
- Block privileged containers and disallowed configurations.
- Enforce image policies, resource limits and security standards.

Admission controls prevent insecure configurations from reaching your cluster.

8. Network Policies



- Network policies control traffic between pods, namespaces and external systems.
- Use a deny-by-default approach.
- Allow only required traffic.
- Segment applications and sensitive workloads.
- Monitor and regularly review network rules.

Network policies reduce lateral movement and limit the impact of a compromise.

9. Cluster Attack Surface



- The Kubernetes control plane, API server, etcd and worker nodes are high-value targets.
- Keep Kubernetes and all components updated.
- Restrict API server access (e.g. no public access).
- Use strong authentication and audit logging.
- Monitor for suspicious activity and misconfigurations.
- Follow CIS Kubernetes Benchmarks.

A secure cluster minimizes the attack surface and improves your ability to detect and respond to threats.



Junior Engineer Tip:

Start with a local cluster (e.g. kind or minikube) and practice applying these security controls.



Remember:

Kubernetes security is a continuous process. Regularly review, update and monitor your cluster.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Kubernetes RBAC and Least Privilege

Control who can do what in your Kubernetes cluster. Use RBAC to grant only the minimum permissions needed. Reduce risk, limit blast radius, and prevent accidental or malicious access.

1. What is Kubernetes RBAC?



- RBAC (Role-Based Access Control) controls who can perform actions on Kubernetes resources.
- It uses Roles, ClusterRoles, RoleBindings and ClusterRoleBindings.
- It follows the principle of least privilege.
- Helps prevent accidental changes and malicious access.

RBAC is essential for a secure and well-governed Kubernetes cluster.

2. Roles (Namespace Scope)



- A Role defines permissions for resources within a specific namespace.
- Used for namespaced resources (e.g. pods, configmaps, secrets).
- Does not grant access outside the namespace.
- Ideal for application teams with limited scope.

Use Roles to give namespace-level access only.

3. ClusterRoles (Cluster Scope)



- A ClusterRole defines permissions for cluster-wide resources or across all namespaces.
- Used for non-namespaced resources (e.g. nodes, persistentvolumes) or when access is needed in multiple namespaces.
- More powerful, so use carefully.

Only use ClusterRoles when cluster-wide access is required.

4. RoleBindings



- A RoleBinding assigns a Role to a user, group, or service account within a namespace.
- It connects the role and the subject (who gets the permissions).
- Used for namespace-scoped access control.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-read-binding
  namespace: dev
subjects:
- kind: ServiceAccount
  name: app-sa
roleRef:
  kind: Role
  name: app-read
  apiGroup: rbac.authorization.k8s.io
```

5. ClusterRoleBindings



- A ClusterRoleBinding assigns a ClusterRole to a user, group, or service account.
- It grants cluster-wide permissions.
- Use with caution to avoid over-permissioning.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: admin-binding
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io
```

6. Service Accounts



- Service accounts provide an identity for applications and pods to access the Kubernetes API.
- They should have only the permissions required for the application to function.
- Avoid using the default service account.
- Each application should have its own dedicated service account with minimal access.

Never give more permissions to a service account than it actually needs.

7. Excessive Permissions



- Overly broad permissions increase the risk of accidental changes and security breaches.
- Avoid using cluster-admin unless absolutely necessary.
- Regularly review and remove unused permissions.
- Use least privilege and follow the principle of "just enough access".

Excessive permissions can turn a small mistake into a major outage or breach.

8. Inspecting Authorization



- Use kubectl to check what a user or service account can do.
- Test permissions before granting them.
- Helps verify least privilege and detect misconfigurations.

```
# Can a user perform an action?
kubectl auth can-i get pods
--as=alice --namespace=dev

# List permissions for a service account
kubectl auth can-i --list
--as=system:serviceaccount:dev:app-sa
```

9. Key Takeaways



- Use Roles for namespace access and ClusterRoles for cluster-wide access.
- Bind roles using RoleBindings and ClusterRoleBindings.
- Use dedicated service accounts with minimal permissions.
- Avoid excessive permissions.
- Regularly review and audit access.
- Use kubectl auth can-i to inspect authorization.

Least privilege is not a one-time task, it's an ongoing practice.



Junior Engineer Tip:

Start with a read-only role, test what you can do, then gradually add the minimum permissions you need.



Remember:

Access you don't control can be abused. Always follow the principle of least privilege.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Kubernetes Workload and Network Security

Run secure workloads and control how they communicate. Reduce the attack surface with least privilege, strong isolation, and proper network controls.

1. Non-Root Workloads



- Do not run containers as root.
- Use a non-root user in your Dockerfile (USER instruction).
- Set runAsNonRoot: true in securityContext.
- Use a specific UID/GID instead of root.
- Reduces the impact if a container is compromised.

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  runAsGroup: 10001
  fsGroup: 10001
```

A non-root container limits damage and improves security.

2. Capabilities



- Linux capabilities give fine-grained privileges to containers.
- Drop all unnecessary capabilities.
- Use only the capabilities your application needs (e.g. NET_BIND_SERVICE for low ports).
- Avoid privileged containers.

```
securityContext:
  capabilities:
    drop:
      - ALL
    add:
      - NET_BIND_SERVICE
```

Use the minimum capabilities required. Drop the rest.

3. Read-Only Filesystems



- Use read-only root filesystems where possible.
- Mount writable paths only when needed (e.g. /tmp).
- Prevents attackers from modifying binaries or configs inside the container.

```
securityContext:
  readOnlyRootFilesystem: true
volumeMounts:
  - name: tmp
    mountPath: /tmp
volumes:
  - name: tmp
    emptyDir: {}
```

A read-only filesystem reduces the chance of runtime tampering.

4. Resource Boundaries



- Set CPU and memory requests and limits.
- Prevents noisy neighbors from affecting other workloads.
- Reduces the risk of resource exhaustion (DoS).
- Helps with stability and predictable performance.

```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

Resource boundaries improve stability and reduce attack impact.

5. Network Policies



- Control which pods can communicate with each other.
- Default to deny, then allow only required traffic.
- Limit east-west traffic between namespaces.
- Reduce lateral movement during a breach.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
spec:
  podSelector: {}
  policyTypes:
    - Egress
```

Network policies restrict traffic and limit the blast radius.

6. Image Trust



- Use trusted and minimal base images.
- Scan images for vulnerabilities (e.g. Trivy, Gripe).
- Use signed images (cosign).
- Avoid using :latest tags in production.
- Use private registries and image policies (e.g. admission controls).

Only run trusted, scanned and signed images.

7. Workload Isolation



- Use namespaces to separate environments (dev, test, prod).
- Apply resource quotas and limits per namespace.
- Use node selectors, taints and tolerations to isolate sensitive workloads.
- Combine with network policies and RBAC.

Isolation reduces the impact of a compromise and limits lateral movement.

8. Common Mistakes to Avoid



- Running containers as root.
- Using privileged containers.
- Not setting resource limits.
- Allowing all network traffic.
- Using untrusted or outdated images.
- Storing secrets in images.
- Not using network policies.
- Overly broad permissions.
- No workload isolation.

Avoid these mistakes to keep your cluster and workloads secure.

9. Key Takeaways



- Run as non-root with least privileges.
- Drop unnecessary capabilities.
- Use read-only filesystems.
- Set resource requests and limits.
- Implement network policies.
- Use trusted and signed images.
- Isolate workloads with namespaces and node controls.
- Security is a layered approach.

Secure workloads + controlled networks = a more resilient Kubernetes cluster.



Junior Engineer Tip:

Start with one workload. Apply non-root, resource limits and a network policy. Test and then make it a standard template for all workloads.



Remember:

Kubernetes security is about reducing risk, not eliminating it. Layer multiple controls and keep improving.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Cloud IAM and Least Privilege

Control who can do what in your cloud environment. Use least privilege, strong authentication and continuous monitoring to reduce risk and prevent compromise.

1. Users (Human Identities)



- Users are individual human identities that access cloud resources.
- Create individual user accounts (do not share accounts).
- Use strong passwords and MFA.
- Use groups for easier permission management.
- Remove inactive users regularly.
- Monitor user activity and access patterns.

Individual users must be accountable.
No shared accounts.

2. Roles (Least Privilege)



- Roles provide temporary permissions instead of long-term credentials.
- Assign only the permissions needed for the job (least privilege).
- Use separate roles for different workloads (e.g. CI/CD, applications).
- Avoid using admin or wildcard permissions (*).
- Regularly review role permissions.

Give the minimum permissions required, no more.

3. Service Identities



- Used by applications, containers, pipelines and automated systems.
- Examples: AWS IAM Roles for EC2/EKS, Azure Managed Identities, GCP Service Accounts.
- Avoid long-term static keys.
- Use workload identity and federation where possible.
- Limit permissions to specific resources and actions.

Service identities should have only the permissions they need.

4. Policies



- Policies define what actions are allowed or denied.
- Use least privilege and resource-level permissions.
- Avoid overly broad permissions (e.g. "*" or "Administrator").
- Use conditions (IP, MFA, resource tags) to add control.
- Keep policies simple and well-documented.
- Regularly audit and remove unused policies.

Well-designed policies reduce risk and prevent accidental access.

5. Temporary Credentials



- Use temporary credentials (STS, federated logins, OIDC) instead of long-term keys.
- Set short expiration times.
- Use role assumption for workloads and automation.
- Avoid hardcoding credentials in code, images or configs.
- Rotate credentials automatically where possible.

Temporary credentials reduce the risk of theft and misuse.

6. Excessive Permissions



- Over-permissioned users, roles or service accounts increase the attack surface.
- Common issues: admin access, wildcards (*), unused permissions, and overly broad policies.
- Regularly review permissions using tools like IAM Access Analyzer, Azure AD, or GCP IAM Recommender.
- Remove permissions that are not needed.

Excessive permissions are one of the top causes of cloud security breaches.

7. Privilege Escalation



- Attackers may exploit overly permissive policies to escalate their privileges.
- Restrict who can create or modify roles and policies.
- Use permission boundaries and service control policies (SCP) where available.
- Monitor for unusual activity and escalation attempts.
- Follow the principle of least privilege at all times.

Preventing privilege escalation keeps your cloud environment secure.

8. Multi-Factor Authentication (MFA)



- Use MFA for all human users, especially admin accounts.
- Enforce MFA through cloud identity providers.
- Use hardware tokens or authenticator apps (e.g. Google Authenticator, Authy, YubiKey).
- Do not rely only on SMS.
- Monitor and alert on failed MFA attempts.

MFA significantly reduces the risk of unauthorized access.

9. Credential Rotation



- Regularly rotate access keys, secrets and tokens.
- Use short-lived credentials where possible.
- Automate rotation with tools and cloud-native features.
- Remove and disable old credentials.
- Monitor for exposed credentials in code, images and repositories.

Regular rotation limits the damage if credentials are exposed.



Junior Engineer Tip:

Always start with least privilege, enable MFA, and review access regularly. Security is a continuous process, not a one-time setup.



Remember:

If you don't control access, you don't control your cloud. Least privilege, monitoring and regular reviews are essential.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Cloud Network and Data Security

Protect your cloud networks and data. Use strong network controls and encryption to keep your applications and information secure.

1. Security Groups (Virtual Firewalls)



- Control inbound and outbound traffic to your resources (e.g. EC2, RDS).
- Allow only required ports and trusted sources.
- Use least privilege rules (e.g. specific IP ranges).
- Remove unused and overly broad rules (0.0.0.0/0).
- Regularly review and audit security group rules.

Security groups are your first line of defense in the cloud.

2. Network Firewall Rules



- Use network ACLs and cloud firewalls for additional control (e.g. AWS NACL, Azure NSG).
- Implement deny-by-default where possible.
- Restrict traffic between subnets and environments (dev, test, prod).
- Log and monitor firewall activity.
- Regularly review and remove unnecessary rules.

Well-defined firewall rules reduce the attack surface.

3. Private Networks



- Use VPCs and private subnets to keep resources off the public internet.
- Access private resources using VPN, Direct Connect or Private Link.
- Use bastion hosts or SSM instead of direct public access.
- Separate environments using subnets and route tables.
- Restrict internet access to only what is necessary.

Keep sensitive resources private and only expose what is required.

4. TLS (Encryption in Transit)



- Use TLS to encrypt data in transit between clients, services and databases.
- Enforce strong TLS versions (e.g. TLS 1.2 or 1.3).
- Avoid weak ciphers and outdated protocols.
- Use certificates from trusted CAs and rotate them regularly.
- Monitor for expired or invalid certificates.

TLS protects your data from eavesdropping and tampering.

5. Encryption at Rest



- Enable encryption for all storage services (e.g. EBS, S3, RDS, Azure Disk).
- Use customer-managed keys (CMK) where required.
- Restrict access to encryption keys using IAM policies.
- Enable automatic key rotation.
- Ensure backups and snapshots are also encrypted.

Encrypt all data at rest, including backups and snapshots.

6. Storage Exposure



- Avoid public access to storage buckets (e.g. S3).
- Use bucket policies and access controls to restrict access.
- Enable block public access settings.
- Regularly scan for exposed storage.
- Use encryption and access logging.

Publicly exposed storage is a major cloud security risk.

7. Databases Security



- Do not expose databases to the public internet.
- Use private subnets and security groups.
- Encrypt data at rest and in transit.
- Use strong authentication and rotate credentials.
- Enable database audit logs and monitoring.

Keep databases private, encrypted and well monitored.

8. Public Endpoints



- Avoid unnecessary public endpoints (e.g. public IPs, open APIs, management interfaces).
- Use load balancers, API gateways and WAF for protection.
- Restrict access by IP, region or authentication.
- Regularly scan for exposed endpoints.
- Disable or remove endpoints that are no longer needed.

Only expose public endpoints when absolutely necessary.

9. Key Takeaways



- Use security groups and firewalls to control traffic.
- Keep resources in private networks.
- Encrypt data both in transit and at rest.
- Avoid public exposure of storage, databases and services.
- Monitor, audit and regularly review your network and data security controls.

Strong network and data security protects your cloud, your data and your business.



Junior Engineer Tip:

Always ask: "Does this need to be public?" If not, keep it private, encrypted and properly controlled.



Remember:

One misconfigured network rule or public endpoint can lead to a major security breach.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

DAST and Runtime Security Testing

Test your running applications for security vulnerabilities. Use DAST and runtime testing to find real-world issues, validate controls, and fix them before attackers do.

1. What is DAST?



- DAST (Dynamic Application Security Testing) tests a running application from the outside.
- It simulates real attacker behavior.
- It does not require access to source code.
- Finds issues like OWASP Top 10 vulnerabilities.
- Common tools: OWASP ZAP, Burp Suite, Nikto, Nuclei.

DAST shows how your application behaves in the real world.

2. Testing Running Applications



- Test the live or staging environment while the application is running.
- Crawl and discover endpoints automatically.
- Test for common vulnerabilities (XSS, SQLi, CSRF, etc.).
- Validate security headers, SSL/TLS configuration and error handling.
- Use authenticated and unauthenticated scans.

Test the application the way an attacker would.

3. Endpoints and Attack Surface



- Discover all endpoints (including hidden ones).
- Test APIs, web forms, file uploads and admin panels.
- Check for sensitive data exposure.
- Identify unnecessary endpoints and HTTP methods.
- Look for insecure configurations and verbose error messages.

The larger your attack surface, the higher the risk.

4. Authentication Considerations



- Test both authenticated and unauthenticated access.
- Use valid test accounts with least privilege.
- Check for broken authentication, session management issues and weak password policies.
- Test for privilege escalation and IDOR (Insecure Direct Object References).
- Avoid brute-force attacks on production systems.

Authentication controls must be tested, not just assumed.

5. Safe Testing Environments



- Always test in a non-production environment (dev, test, or staging).
- Get proper authorization before testing.
- Avoid actions that can disrupt services (e.g. DoS, destructive payloads).
- Use rate limiting and safe scan policies.
- Sanitize test data and follow responsible testing practices.

Test safely, legally and responsibly.

6. Common Findings



- Injection vulnerabilities (SQLi, NoSQLi, command injection).
- Cross-Site Scripting (XSS).
- Cross-Site Request Forgery (CSRF).
- Security misconfigurations and missing headers.
- Exposed sensitive data and credentials.
- Broken access controls and excessive permissions.

DAST helps you find real, exploitable vulnerabilities in running applications.

7. Runtime Security Testing



- Monitor application behavior during runtime.
- Use runtime security tools (e.g. RASP, container/runtime scanners).
- Detect abnormal activity, malicious payloads and unexpected process behavior.
- Combine DAST with runtime monitoring for better coverage.
- Useful in containers, Kubernetes and cloud environments.

Runtime testing detects threats that traditional scans may miss.

8. Remediation and Verification



- Prioritize findings based on risk (critical, high, medium, low).
- Fix vulnerabilities and misconfigurations.
- Apply secure configuration and patches.
- Re-test after fixes to confirm the issue is resolved.
- Integrate DAST into CI/CD pipelines for continuous testing.

Finding vulnerabilities is not enough. Fix them and verify the fix.

9. Key Takeaways



- DAST tests running applications from the outside.
- Use safe, non-production environments.
- Test endpoints, authentication and configurations.
- Review and remediate findings.
- Combine DAST with runtime security for stronger protection.
- Make security testing a continuous part of your DevSecOps process.

Test. Fix. Verify. Repeat.
A more secure application is a stronger business.



Junior Engineer Tip:

Start with OWASP ZAP in a safe environment. Learn how to scan, read the report, fix an issue, and re-test it.



Remember:

Never run aggressive scans on production systems. Always get approval and use authorized testing environments.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Security Logging, Monitoring, and Detection

Collect, monitor, and analyze security data from your applications, infrastructure and cloud to detect threats early and take action.

1. Application Logs



- Log important application events (errors, warnings, authentication, admin actions).
- Include timestamps, user identity and request IDs.
- Avoid logging sensitive data (passwords, tokens, PII).
- Use structured logging (JSON) for easier analysis.
- Send logs to a centralized system (e.g. ELK, CloudWatch, Azure Monitor).

Good application logs help you detect issues and security threats early.

2. Authentication Events



- Monitor successful and failed login attempts.
- Track MFA usage and failures.
- Look for unusual login locations, times or devices.
- Detect repeated failures and possible brute force attacks.
- Alert on privilege escalations and new account creations.

Authentication logs show who is accessing your systems.

3. Cloud Audit Logs



- Enable and monitor cloud audit logs (e.g. AWS CloudTrail, Azure Activity Logs, GCP Audit Logs).
- Track changes to IAM, networking, storage and security policies.
- Look for unusual or unauthorized API calls.
- Detect creation or deletion of critical resources.
- Send logs to a centralized SIEM or logging platform.

Cloud audit logs give you visibility into everything happening in your cloud.

4. Kubernetes Audit Data



- Enable Kubernetes audit logs on the API server.
- Monitor access to sensitive resources (pods, secrets, configmaps, RBAC).
- Track create, delete, update operations.
- Look for unusual or unauthorized access patterns.
- Send audit logs to a centralized logging solution.

Kubernetes audit logs help you detect suspicious activity in your cluster.

5. Security Alerts



- Set up alerts for critical security events (failed logins, policy changes, unusual activity).
- Use threshold-based and behavior-based alerts.
- Reduce noise by tuning alert rules.
- Integrate alerts with chat tools (e.g. Slack, Teams).
- Ensure alerts are actionable and assigned to the right team.

Good alerts help you respond before small issues become big breaches.

6. SIEM Concepts



- SIEM (Security Information and Event Management) collects and analyzes logs from multiple sources.
- Correlates events to detect threats and patterns.
- Provides dashboards, alerts and reporting.
- Common tools: Splunk, ELK (Elastic), Azure Sentinel, IBM QRadar, Google SecOps.
- Helps with compliance and incident response.

A SIEM gives you a single view of your security data.

7. Suspicious Behavior



- Look for unusual login times, locations or devices.
- Detect access from unknown IP addresses or countries.
- Identify privilege escalation attempts.
- Monitor unexpected changes to critical resources.
- Look for lateral movement and data exfiltration patterns.

Suspicious behavior is often the first sign of a real attack.

8. Actionable Alerts



- Focus on high-risk, actionable alerts (not just low-risk noise).
- Include clear context (what happened, where, who, and recommended action).
- Automate response where possible (e.g. block IP, disable account).
- Define an incident response process for alert handling.
- Regularly review and improve alert rules.

An alert is only useful if it leads to action.

9. Key Takeaways



- Log everything that matters (applications, cloud, Kubernetes).
- Monitor authentication and access events.
- Use a SIEM or centralized logging platform.
- Set up meaningful, actionable alerts.
- Look for suspicious behavior and respond quickly.
- Continuously tune and improve your logging and monitoring.

Visibility + Monitoring + Action = A more secure environment.



Junior Engineer Tip:

Start with logging and monitoring from day one. You cannot detect what you do not record.



Remember:

Logs, monitoring and alerts are not just for compliance. They help you detect and stop real attacks.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Vulnerability Management in Production

Find, assess, prioritize, fix and continuously track vulnerabilities in your production environment based on real risk, not just severity.



1. What is Vulnerability Management?



- A continuous process to identify, assess, prioritize, remediate and track vulnerabilities.
- Covers applications, containers, infrastructure, cloud services and third-party dependencies.
- Reduces the risk of real attacks in production.

It's not just scanning — it's about managing risk until it is closed.

2. CVE (Common Vulnerabilities and Exposures)



- CVE is a unique public ID for a known vulnerability (e.g. CVE-2024-3094).
- Provides a standard way to reference vulnerabilities across tools and vendors.
- Contains a short description but not full technical details.
- Used in security advisories, scanners and threat intelligence.

CVE tells you what the vulnerability is, not how serious it is.

3. CVSS (Common Vulnerability Scoring System)



- CVSS provides a base score from 0.0 to 10.0.
- 0.0 – 3.9 = Low
- 4.0 – 6.9 = Medium
- 7.0 – 8.9 = High
- 9.0 – 10.0 = Critical
- Scores are based on technical factors (exploitability, impact, etc.).
- Does not consider your specific environment or asset value.

CVSS helps you understand severity, but it's not the full risk picture.

4. Exploitability



- Exploitability means how easy it is for an attacker to exploit the vulnerability.
- Consider if there is a public exploit, active attacks or exploit code available.
- Look at attacker effort, required privileges and user interaction.
- Vulnerabilities with active exploits should be prioritized higher.

A low CVSS score can still be high risk if it is easy to exploit.

5. Asset Criticality



- Not all assets are equal.
- A vulnerability on a public-facing production system is more critical than on a test environment.
- Consider business impact, data sensitivity and dependencies.
- Classify assets (e.g. critical, high, medium, low).
- Use asset inventory and CMDB where possible.

The same vulnerability can be low risk or critical depending on the asset.

6. Exposure



- Consider where the vulnerable asset is exposed.
- Is it publicly accessible (internet facing) or internal only?
- Check network access controls, firewalls and security groups.
- Look at whether authentication is required.
- Reduce unnecessary exposure (e.g. close unused ports and services).

The more exposed an asset, the higher the real risk.

7. Remediation SLAs



- Set clear SLAs based on risk severity and asset criticality.
- Example SLAs:
 - Critical: 7 days
 - High: 14 days
 - Medium: 30 days
 - Low: 90 days
- Track progress and escalate overdue items.
- Document exceptions with justification.

SLAs keep remediation accountable and reduce long-term risk.

8. Why Severity Alone is Not Enough



- A high CVSS score on a non-exposed test system may be lower risk than a medium score on a public production system.
- Real risk = Severity + Exploitability + Asset Criticality + Exposure + Business Impact.
- Consider threat intelligence, active attacks and compensating controls.
- Focus on what attackers can actually reach and exploit in your environment.

Severity is just one piece of the puzzle.

9. Key Takeaways



- Follow the full lifecycle: Discover → Validate → Prioritize → Assign → Remediate → Verify → Track.
- Understand CVE, CVSS, exploitability, asset criticality and exposure.
- Set and enforce remediation SLAs.
- Focus on real risk, not just severity.
- Continuously monitor and improve your process.

Effective vulnerability management keeps your production environment secure and resilient.



Junior Engineer Tip:

Don't just fix the high scores. Understand the risk, ask questions, and always think like an attacker.



Remember:

A vulnerability is not a real risk until it can be exploited in your environment.



**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Security Incident Response for DevSecOps

Detect, triage, contain, preserve evidence, eradicate, recover, verify and learn.
Be prepared, respond fast, and turn incidents into a stronger and more secure environment.



1. Detect



- Identify suspicious activity (e.g. leaked credentials, unauthorized access, unusual behavior).
- Use monitoring, alerts, code scanning and community reports.
- Sources: GitHub, CI/CD logs, cloud logs, SIEM alerts, user reports.

Detect early to reduce impact.

2. Triage



- Confirm it is a real incident (false positive or real?).
- Assess scope and severity.
- Identify affected systems, accounts and data.
- Determine if the incident is active right now.
- Classify the incident (e.g. credential leak, malware, data exposure).

Understand what you are dealing with.

3. Contain



- Take immediate action to limit further damage.
- Revoke exposed credentials.
- Disable or restrict affected accounts and access.
- Block malicious IPs or traffic.
- Isolate affected systems (e.g. remove from network).
- Do not wait for full analysis if the risk is high.

Stop the bleeding first.

4. Preserve Evidence



- Collect and secure logs, artifacts and system data.
- Take snapshots (VMs, containers, configs).
- Preserve Git commits, CI/CD logs and timestamps.
- Maintain chain of custody (for legal or compliance).
- Do not modify or delete evidence.

Evidence is critical for investigation.

5. Eradicate



- Remove the root cause (e.g. malicious code, backdoors, unauthorized keys).
- Clean compromised systems and accounts.
- Patch vulnerabilities.
- Remove exposed secrets from code and repositories (history and forks).
- Ensure no malicious persistence remains.

Remove the threat completely.

6. Recover



- Restore services safely (e.g. from clean backups).
- Rotate all affected credentials (API keys, tokens, passwords).
- Redeploy applications with fixed configurations.
- Monitor closely for signs of continued compromise.
- Validate that systems are working as expected.

Bring services back in a secure way.

7. Verify



- Confirm the threat is removed.
- Check for any remaining vulnerabilities.
- Monitor logs and alerts for unusual activity.
- Validate security controls (e.g. access, network, IAM).
- Get sign-off from the incident response team.

Make sure it's really resolved.

8. Learn



- Conduct a post-incident review (what happened, why, and what to improve).
- Update security policies, processes and tooling.
- Add automated controls to prevent recurrence (e.g. secret scanning, pre-commit hooks, CI/CD checks).
- Share lessons with the team.
- Turn incidents into a stronger defense.

Every incident is a learning opportunity.

Scenario: Public Credential in a Git Repo



A production credential appears in a public repository. What should the engineer do first?

- A. Analyze who accessed it
- B. Revoke the credential immediately ✓
- C. Wait and monitor for unusual activity
- D. Close the repository

Correct Answer: B
Revoke the credential immediately.



Junior Engineer Tip:

If you ever suspect a credential is exposed, revoke it immediately, then investigate. Always assume it is already compromised.



Remember:

In security incidents, speed, containment and clear communication can prevent a small issue from becoming a major breach.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL

Build a Complete DevSecOps Pipeline

Integrate security into every stage of your software delivery pipeline.
Build secure, reliable and production-ready applications.



Key Takeaways

- Security must be integrated at every stage.
- Multiple controls address different threats.
- Evidence is important for compliance and audits.
- A failed control should stop the pipeline or trigger remediation.
- Continuous feedback makes the pipeline stronger.

Final Challenge for the Learner

1. Explain what security control exists at every stage.
2. Identify the threat it addresses.
3. State what evidence it produces.
4. Define what should happen when the control fails.
5. Be ready to discuss trade-offs and real-world implementation challenges.

Remember

- Security is everyone's responsibility.
- Automate as much as possible.
- Fail fast and fix early.
- Keep evidence for compliance.
- A secure pipeline delivers safer software to production.



Junior Engineer Tip:

Start small. Implement one security control at a time, then gradually build the full pipeline.



Real-World Mindset:

Security is not a one-time activity — it's a continuous process. Build, measure, learn and improve.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

**SAME DISCIPLINE
HIGHER IMPACT**

EDUCATE • EMPOWER • ENGINEER • EXCEL